RiskTrix

Technical Documentation

A complete guide to the platform — architecture, services, modules, and features.

Version 1.0 · May 29, 2026 · Internal Use Only

11	30+	100+	7
Microservices	Pages	API Endpoints	User Roles

What is this document?

This document explains everything about the RiskTrix platform in plain language. It covers what RiskTrix does, how it is built, every feature a user can access, how the backend services work, and how data flows through the system. Whether you are a brand new user, a developer, or a manager — this guide is written so that anyone can understand how RiskTrix works without needing prior technical knowledge.

Who should read which section?

New users: Start with Section 1 (What is RiskTrix?) then Section 5 (Platform Modules) for feature walkthroughs. Developers: Focus on Section 2 (Architecture), Section 6 (Microservices), and Section 10 (Developer Map). Managers and auditors: Read Section 1, Section 3 (Roles), Section 5 (Features), and Section 8 (Workflows).

© 2026 RiskTrix · Confidential — Internal Use Only

Table of Contents

Every section in this document

1.	**What is RiskTrix?**	*The product explained — purpose, users, and value*
2.	**System Architecture**	*How all services and components fit together*
3.	**User Roles and Permissions**	*Who can access what, and why*
4.	**The Frontend Application**	*How the web application is structured and navigated*
5.	**Platform Modules — Full Feature Guide**	*Every sidebar module with all features explained*
5.1	Dashboard	
5.2	Asset Management	
5.3	Policies and Device Management	
5.4	Tickets	
5.5	Security	
5.6	Automation Engine	
5.7	File Manager	
5.8	Agent Directory	
5.9	Search	
5.10	Queue Monitor	
5.11	Settings	
5.12	User Management	
6.	**Backend Services In Detail**	*Each microservice — what it does, its API, its behavior*
7.	**Data and Database**	*What data is stored and in which service*
8.	**End-to-End Workflows**	*Step-by-step walkthroughs of key platform operations*
9.	**Deployment and Local Setup**	*How to run RiskTrix on your machine*
10.	**Developer Quick-Start Map**	*Where to look first when reading the codebase*
11.	**Glossary**	*Plain-English definitions of every technical term*

© 2026 RiskTrix · Confidential — Internal Use Only

Introduction

How to Read This Document

A guide for every type of reader

This document is written for any reader — developer, manager, auditor, or first-time user — who needs to understand how RiskTrix works. You do not need any prior technical knowledge. Technical words are explained when they first appear, and every section starts with a plain-language explanation before going into detail.

I am completely new to RiskTrix

Start with Section 1 — What is RiskTrix? — to understand what the product does and who it is for. Then read Section 2 to see how all the pieces connect. Then go to Section 5, which walks through every feature in the application, module by module.

I am a developer joining the team

Read Section 2 for the overall architecture and technology stack, Section 6 for each backend service's API and behaviour, and Section 10 for the fastest path into the codebase.

I am a manager, auditor, or security reviewer

Focus on Section 1 for the product overview, Section 3 for roles and access control, Section 5 for what each module does, and Section 8 for how the key security workflows operate end to end.

© 2026 RiskTrix · Confidential — Internal Use Only

Section 2

How RiskTrix is Built

Architecture in plain English

RiskTrix is built as a microservices platform. This means the system is not one large application — it is a collection of smaller, focused services that each own one area of the product. They talk to each other over standard web APIs.

What is a microservice?

Imagine a large company with separate departments — HR, Finance, IT, Legal. Each department has its own responsibilities and its own records. If Finance needs information about an employee, it calls HR. Microservices work the same way. The Tickets service handles only tickets. The Assets service handles only assets. They call each other when they need data from another area.

The three layers of RiskTrix:

1

The Frontend — what you see in your browser

A React web application that runs at port 8888. It shows all the dashboards, tables, forms, and charts you interact with. It does not store data itself — it fetches everything from the backend services via APIs.

2

The Backend Services — the business logic

Ten separate FastAPI services, each responsible for one domain. They validate your input, apply business rules, and save results to the database. The main backend handles login, users, and shared platform features. Specialist services handle assets, tickets, policies, security, files, search, and more.

3

The Database and Infrastructure — the memory

PostgreSQL stores all platform data. Redis is available for caching and event-based communication. An Nginx API Gateway acts as the single entry point for requests from outside. Docker Compose starts everything together with a single command.

How a typical request travels through the system — step by step:

1

Open the browser

The React frontend loads. You see the login screen.

2 **Log in**

Your username and password are sent to the Main Backend, which checks them and returns a token — a small digital proof that you are who you say you are. The frontend stores this token and sends it with every future request.

3 **Navigate to a page**

The frontend checks your role and shows only the pages you are permitted to access. Menu items you do not have permission for are hidden.

4 **Load data**

The page makes an API call to the relevant microservice — for example, the Asset Service when you open Asset Management. The service queries the database and returns the data.

5 **See the results**

The frontend renders the data as tables, charts, and cards.

6 **Save a change**

When you fill in a form and click Save, the frontend sends a POST or PUT request. The service validates it, saves it to the database, and may trigger background events for other services to process.

© 2026 RiskTrix · Confidential — Internal Use Only

Section 3

Authentication and RBAC

RiskTrix uses role-based access control. The frontend permission matrix is defined in cisodashboard-frontend/src/utils/permissions.js; backend checks also exist in cisodashboard-backend/app/core/rbac.py and service-level dependencies.

3.1 Roles

What does a role mean in practice?

A role is a label that controls what a user is allowed to see and do. For example, a Viewer can only read data and cannot make changes. A Security Analyst can create alerts and remediation tasks. A CISO can see everything and change system settings. Roles are assigned when an account is created by an administrator.

Role	Label	Abbreviation
`admin`	Administrator	ADM
`ciso`	CISO	CSO
`infosec_manager`	InfoSec Manager	MGR
`security_analyst`	Security Analyst	ANA
`internal_auditor`	Internal Auditor	INT
`external_auditor`	External Auditor	EXT
`viewer`	Viewer	VWR

3.2 Role Inheritance

How role inheritance works

Inheritance means a senior role automatically gets all the permissions of roles below it in the chain. So a CISO can do everything an InfoSec Manager can do, plus more. Auditor roles are intentionally kept separate from this chain — they can see compliance and audit data but are blocked from operational management screens like the Agent Directory.

The primary inheritance chain is:

admin -> ciso -> infosec_manager -> security_analyst -> viewer

Auditor roles are intentionally separate:

- internal_auditor does not inherit viewer
- external_auditor does not inherit viewer

This separation allows the platform to show audit/compliance views to auditors while hiding operational pages such as Agent Directory.

3.3 Sidebar Visibility For Documented Modules

Sidebar Module	Visibility
Dashboard	All authenticated users
Asset Management	Users with `assets.view`
Policies	Users with `policies.view`
Tickets	Users with `tickets.view`
Security	Users with `vulnerabilities.view`
Automation	InfoSec Manager, CISO, Admin
File Manager	Users with `files.view`
Agent Directory	Viewer, Security Analyst, InfoSec Manager, CISO, Admin
Search	All authenticated users
Queue Monitor	Internal Auditor, External Auditor, InfoSec Manager, CISO, Admin
Settings	CISO, Admin

Section 4

Frontend Application Structure

The frontend entry point is cisodashboard-frontend/src/App.jsx. It defines public routes, authenticated routes, admin-only routes, and permission-guarded routes.

Important frontend areas:

- src/components/Navigation/Sidebar.jsx: sidebar module and child menu definitions.
- src/components/layouts/MainLayout.jsx: application shell layout.
- src/context/RBACContext.jsx: role/permission context.
- src/utils/api.js: default backend API client.
- src/utils/*Api.js: module-specific API clients for assets, tickets, policies, directory, files, queue, search, security, automation, commands, devices, and heartbeat.
- src/pages/*: page implementations for each documented sidebar module.

Section 5

Sidebar Modules and Functionality

5.1 Dashboard

Your Personal Home Screen

When you first log in to RiskTrix, the Dashboard is what you see. It is your personal starting point — showing work that has been assigned to you, a feed of recent activity across the platform, and quick links to the areas you work in most. Think of it as the morning briefing screen that tells you what needs your attention today.

Route: /

Purpose: User home screen. It shows assigned work, recent activities, and a quick overview of the user's current responsibilities.

Primary backend areas:

- /api/v1/yourboard
- /api/v1/activities
- Dashboard/shared services in cisodashboard-backend/app/modules/shared

Main functionality:

- Displays user-specific board items.
- Displays activity feed.
- Gives a quick entry point into pending operational work.

5.2 Asset Management

Track Everything Your Organisation Owns

Before you can protect something, you need to know it exists. Asset Management is where you register and monitor every device, system, and application in your organisation — servers, laptops, mobile devices, web applications, cloud services, databases, and more. Each asset has a risk level, a list of known vulnerabilities, a software inventory, attached files, and a full change history. If something changes on an asset, you can see exactly what changed and when.

Sidebar children:

Page	Route	Purpose
Asset Inventory	`/assets`	List, search, create, update, import, and inspect assets
Asset Dashboard	`/assets/dashboard`	Asset totals, types, status breakdowns, and recent additions

Primary service: asset-service

Core entities:

- Asset
- Device
- Enrollment token/history
- Device command

Main functionality:

- Manual asset creation and update.
- Bulk upload from spreadsheet/CSV.
- Asset detail page through /assets/:id.
- Device enrollment and check-in support.
- Device command tracking for enrolled devices.

Important API groups:

- /api/v1/assets
- /api/v1/assets/import
- /api/v1/assets/dashboard
- /api/v1/enrollment/*
- /api/v1/devices/*

5.3 Policies

Define the Security Rules Every Device Must Follow

Policies are the rules your organisation sets for how devices must be configured to be considered secure. For example: every device must have a screen lock, encryption must be turned on, the operating system must be kept up to date. In this module you create those rules, assign them to specific devices or groups of devices, and run compliance scans to check which devices are following the rules. Devices that fail a check show up as non-compliant so you can take action. You can also send remote commands to managed devices directly from the platform — like locking a lost phone or wiping a compromised laptop.

Sidebar children:

Page	Route	Purpose
Policy Dashboard	`/policies/dashboard`	Overall policy/compliance health
Policy Management	`/policies`	CRUD for policies
Policy Assignments	`/policies/assignments`	Policy-to-device assignment status
Bulk Assign	`/policies/bulk-assign`	Assign many policies to many devices
Conflict Report	`/policies/conflicts`	Detect conflicting policies
Policy Templates	`/policies/templates`	Apply prebuilt policy templates
Compliance Dashboard	`/compliance`	Compliance scan summary
Device Groups	`/groups`	Static/smart grouping of devices
Devices	`/devices`	Device inventory and status
Enrollment Dashboard	`/enrollment`	Device enrollment overview
Enroll Device	`/enrollment/enroll`	Device enrollment form

Primary service: policy-service

Core entities:

- Policy
- Policy assignment
- Device
- Device group
- Compliance scan
- Security alert
- Device security profile
- Remediation action
- Device command

Main functionality:

- Create and manage security policies.
- Assign policies to devices or groups.
- Retry failed assignments.
- Scan policy compliance.
- Detect policy conflicts.
- Track devices and device groups.
- Support MDM-style commands such as lock, wipe, restart, sync, message, clear passcode, and lost mode.

Important API groups:

- /api/v1/policies
- /api/v1/policies/assign/bulk
- /api/v1/groups
- /api/v1/devices
- /api/v1/compliance
- /api/v1/agent/checkin
- /api/v1/devices/{device_id}/commands

5.4 Tickets

> **Manage Every Security Incident and Service Request**
>
> When something needs to be investigated or fixed — a security alert, an access problem, a user complaint, a vulnerability that needs patching — a Ticket is created to track it from start to finish. Each ticket has a priority, a status, an assigned person, notes, attachments, and an SLA timer that shows how long is left before the issue must be resolved. Tickets are automatically assigned to the right person based on category and who has the lightest workload at that moment. Managers can use the Ticket Dashboard to see the health of the whole queue at a glance.

Sidebar children:

Page	Route	Purpose
All Tickets	`/tickets`	Ticket list, filtering, creation, updates, comments, assignment
Ticket Dashboard	`/tickets/dashboard`	Ticket KPIs and recent ticket metrics
Ticket Assignments	`/tickets/assignments`	Agent workload and unassigned ticket view
Assignments	`/tickets/assignments`	Duplicate sidebar label pointing to the same assignment dashboard

Primary service: ticket-service

Core entities:

- Ticket
- Ticket comment
- Ticket attachment
- Ticket link
- SLA metadata

- Outbox event
- User reference

Main functionality:

- Create tickets.
- Filter by status, priority, category, channel, assignment, or search.
- Assign or auto-assign tickets.
- Change status, resolve, close, and bulk update.
- Add comments and upload attachments.
- Track SLA due dates and breaches.
- Export ticket data.

Auto-assignment flow:

1. A ticket is created without assigned_to.
2. Ticket service derives team/tier from ticket category/priority.
3. Ticket service calls Directory Service lookup.
4. Candidate agents are compared by open ticket workload.
5. The least-loaded matching agent is assigned.
6. Notification Service can send assignment emails.

Important API groups:

- /api/v1/tickets
- /api/v1/tickets/dashboard
- /api/v1/tickets/assignment-stats
- /api/v1/tickets/workload
- /api/v1/tickets/{ticket_id}/comments
- /api/v1/tickets/{ticket_id}/attachments
- /api/v1/analytics/*

5.5 Security

> **See the Security Risk of Every Managed Device in Real Time**
>
> The Security module gives you a live view of the risk level of every device the platform manages. It works by calculating a risk score for each device based on security findings — is the device rooted or jailbroken? Is encryption turned off? Is the OS outdated? Are any dangerous apps installed? The higher the score, the higher the risk. When a device crosses a risk threshold, the platform automatically creates a Security Alert. You can acknowledge alerts, assign remediation tasks, and send remote commands like Lock or Wipe from the device's security detail page.

Sidebar children:

Page	Route	Purpose
Security Dashboard	`/security`	Device risk, alerts, risky devices, trends
Submit Agent Report	`/security/report`	Submit device security scan/profile data

Primary service: policy-service

Core entities:

- Device security profile
- Security alert
- Remediation action
- Device command

Main functionality:

- Summarizes device risk levels.
- Lists active alerts and risky devices.
- Accepts agent reports containing rooted/encryption/screen-lock/OS/app-risk findings.
- Recalculates risk scores.
- Creates alerts when thresholds or findings are detected.
- Supports acknowledgement, resolution, remediation, and manual risk evaluation.

Important API groups:

- /api/v1/security/summary
- /api/v1/security/alerts
- /api/v1/security/risky-devices
- /api/v1/security/trend
- /api/v1/security/profile/{device_id}
- /api/v1/security/evaluate/{device_id}
- /api/v1/security/remediation/{device_id}

5.6 Automation

Make the Platform Respond to Security Events Automatically

The Automation Engine lets you write rules that make the platform react to events without anyone having to watch and respond manually. For example, you could write a rule that says: whenever a device risk score goes above 80, automatically lock the device AND create a high-severity alert AND log a remediation action. You build these rules using a simple visual form — no coding required. The engine evaluates every security event, device check-in, and offline notification against your active rules and fires the appropriate actions instantly.

Sidebar children:

Page	Route	Purpose
Automation Rules	`/automation`	List rules, toggle status, view executions and stats
New Rule	`/automation/new`	Build/test automation rules

Primary services:

- Main backend automation module: cisodashboard-backend/app/modules/automation
- Policy service automation routes: policy-service/app/api/routes/automation.py

Core entities:

- Automation rule
- Automation execution

Rule structure:

- Trigger: risk_evaluated, device_offline, agent_checkin, or manual
- Conditions: field/operator/value filters
- Actions: create alert, send notification, change severity, assign ticket, remediation notes, or other supported action types

Important API groups:

- /api/v1/automation/rules
- /api/v1/automation/rules/{rule_id}
- /api/v1/automation/rules/{rule_id}/toggle
- /api/v1/automation/rules/{rule_id}/test
- /api/v1/automation/executions
- /api/v1/automation/stats

5.7 File Manager

One Central Place for All Evidence and Attachments

Every file uploaded anywhere in the platform — a ticket attachment, a vulnerability screenshot, an audit document, an asset manual, a proof-of-concept — is stored and accessible through the File Manager. Files are tagged with a context so you can filter and find everything related to a particular ticket, asset, or audit engagement without searching through multiple modules.

Route: /files

Primary service: file-service

Purpose: Central file storage and retrieval for files attached to platform entities.

Supported contexts:

Context	Typical Use
`ticket`	Ticket attachments
`vulnerability`	Vulnerability evidence
`poc`	Proof-of-concept or closure evidence
`audit`	Compliance/audit documents
`asset`	Asset photos/specifications
`general`	General shared files

Main functionality:

- Upload files with context and optional reference ID.
- Browse files by context/reference.
- Download files by ID.
- Delete files when permitted.

Important API groups:

- /api/v1/files
- /api/v1/files/upload
- /api/v1/files/download/{file_id}

5.8 Agent Directory

The Directory of People Who Handle Security Work

The Agent Directory is the list of people in your team who handle tickets, respond to incidents, and cover on-call shifts. It stores each person's name, email, team, skills, availability schedule, and current ticket workload. When a new ticket arrives without an assigned person, the platform uses this directory to automatically find the most suitable available person and assign it to them — based on matching skills, team, and who currently has the fewest open tickets.

Route: /directory

Primary service: directory-service

Purpose: Directory of support/security agents used by ticket assignment and operational workflows.

Core entities:

- Agent
- Availability
- On-call schedule
- Performance metric

Main functionality:

- Create, list, update, and delete agents.
- Lookup agents by team/tier for ticket auto-assignment.
- Track availability and on-call schedules.
- Provide metrics, summaries, and leaderboards.

Important API groups:

- /api/v1/directory/agents
- /api/v1/directory/agents/lookup
- /api/v1/directory/agents/{agent_id}/availability
- /api/v1/directory/on-call
- /api/v1/directory/metrics
- /api/v1/directory/agents/{agent_id}/metrics

5.9 Search

Find Anything Across the Entire Platform in Seconds

Instead of switching between modules to find something, the Search module lets you type a keyword and instantly see matching results from all areas — tickets, assets, agents, and policies — grouped by type. It is the fastest way to find something when you are not sure which module it belongs to.

Route: /search

Primary service: search-service

Purpose: Cross-service search for users who do not know which module owns the record.

Main functionality:

- Query all supported domains at once.
- Return grouped results such as tickets, assets, agents, and policies.
- Allow resource-specific searches.

Important API groups:

- /api/v1/search
- /api/v1/search/tickets
- /api/v1/search/assets
- /api/v1/search/agents
- /api/v1/search/policies

5.10 Queue Monitor

> **Monitor Background Jobs and Fix Failures**
>
> Some platform operations happen in the background — notifications being sent, events being processed between services. The Queue Monitor shows the status of all these background jobs. If a job failed, authorised users can see exactly what went wrong and replay it. This is mainly used by system administrators and auditors who need to verify that all background processing completed successfully.

Route: /queue

Primary service: queue-service

Purpose: Monitor and manage transactional outbox/event delivery.

Core entity:

- Outbox event

Main functionality:

- List events and filter by status.
- View payload and failure details.
- View queue statistics.
- Replay failed events by resetting them for processing.
- Queue worker polls pending events and attempts delivery.

Important API groups:

- /api/v1/queue/events
- /api/v1/queue/stats
- /api/v1/queue/events/{event_id}/replay

5.11 Settings

> **System Configuration for Administrators**
>
> The Settings module lets administrators manage their own profile, configure compliance standards the platform tracks, and set up email delivery so the platform can send notifications when events occur.

Route: /settings

Primary backend areas:

- Users/auth APIs in cisodashboard-backend/app/modules/shared/api
- Notification/SMTP configuration patterns

Purpose: System-level configuration.

Tabs and functions:

Tab	Purpose
Profile	View and manage profile/user information
Standards	Add/manage compliance standards
SMTP	Configure notification email delivery

5.12 User Management

> **Create and Manage Every User Account**
>
> The User Management page is only accessible to administrators. It shows every user account in the platform and lets admins create new accounts, edit roles and details, activate or deactivate users, and delete accounts that are no longer needed. This is how you control who can log in and what they are allowed to do.

Route: /admin/users

This is not shown as a normal sidebar item. It is admin-only.

Purpose: Full user lifecycle management.

Main functionality:

- Search and filter users.
- Create users.
- Edit user profile, role, privilege level, clearance level, department, company, and phone.
- Activate/deactivate users.
- Delete users.
- Manage roles through admin-only access.

Important API groups:

- /api/v1/users
- /api/v1/auth

Section 6

Main Backend Documentation

Location: cisodashboard-backend

The main backend is a FastAPI application that owns shared platform concerns for authentication, users, activity feeds, dashboard support, settings, and administration.

6.1 Backend Modules In Scope

Module	Location	Responsibility
Shared	`app/modules/shared`	Auth, users, dashboard support, reminders, activities, yourboard
Automation	`app/modules/automation`	Automation rules and execution history

6.2 Shared Module

The shared module contains platform-wide concerns:

- Authentication and token handling
- User management
- Dashboard support
- Activity logs
- YourBoard user work feed
- Reminder models and responses

6.3 Automation Module

The automation module contains:

- Rule tables and execution tables
- Rule CRUD
- Rule testing
- Toggle enable/disable
- Execution history
- Stats endpoint

Section 7

Microservice Documentation

7.1 Asset Service

Location: asset-service

Purpose: Owns assets, asset dashboards, device enrollment, and device command support.

Key folders:

- app/api/routes
- app/models
- app/schemas
- app/services
- app/db

Main API groups:

- /api/v1/assets
- /api/v1/assets/import
- /api/v1/assets/dashboard
- /api/v1/enrollment
- /api/v1/devices
- /health

Important behavior:

- Provides asset inventory and detail records.
- Supports bulk asset upload.
- Provides enrollment token/history endpoints.
- Handles device check-in and device commands.

7.2 Ticket Service

Location: ticket-service

Purpose: Owns ticket lifecycle, assignment, SLA, comments, attachments, analytics, and outbox event creation.

Key folders:

© 2026 RiskTrix · Confidential — Internal Use Only

- app/api/routes
- app/models
- app/schemas
- app/services
- app/core
- app/db

Main API groups:

- /api/v1/tickets
- /api/v1/tickets/dashboard
- /api/v1/tickets/workload
- /api/v1/tickets/assignment-stats
- /api/v1/analytics

Important behavior:

- Auto-assigns tickets using Directory Service.
- Sends notification requests using Notification Service.
- Tracks SLA deadlines and breach state.
- Writes outbox events for asynchronous workflows.

7.3 Directory Service

Location: directory-service

Purpose: Owns the operational agent directory and availability model.

Main API groups:

- /api/v1/directory/agents
- /api/v1/directory/agents/lookup
- /api/v1/directory/agents/{agent_id}/availability
- /api/v1/directory/agents/available-now
- /api/v1/directory/on-call
- /api/v1/directory/metrics
- /health

Important behavior:

- Provides ticket auto-assignment candidates.
- Stores team, tier, skills, timezone, and active status.
- Tracks availability windows and on-call schedules.
- Provides performance summaries and leaderboards.

7.4 Policy Service

Location: policy-service

Purpose: Owns policy management, policy assignment, compliance scanning, device security risk, commands, agent check-in, and automation.

Main API groups:

- /api/v1/policies
- /api/v1/policies/templates
- /api/v1/policies/conflicts
- /api/v1/policies/assign/bulk
- /api/v1/groups
- /api/v1/devices
- /api/v1/compliance
- /api/v1/security
- /api/v1/agent
- /api/v1/automation
- /api/v1/devices/{device_id}/commands

Important behavior:

- Runs policy and compliance logic.
- Computes risk scores from security profiles.
- Creates security alerts.
- Manages remediation actions.
- Supports device commands.
- Supports automation rule execution.

7.5 File Service

Location: file-service

Purpose: Owns file upload, file index metadata, browsing, download, and deletion.

Main API groups:

- /api/v1/files
- /api/v1/files/upload
- /api/v1/files/download/{file_id}
- /health

Important behavior:

- Stores files under context folders.
- Maintains file metadata.
- Allows module-specific file linking by context and reference ID.

7.6 Notification Service

Location: notification-service

Purpose: Owns outbound email notification behavior.

Main API groups:

- /api/v1/notify
- /health

Important behavior:

- Sends or logs email depending on configuration.
- Used by ticket assignment and operational reminder workflows.
- Centralizes mail configuration so other services do not need SMTP details.

7.7 Queue Service

Location: queue-service

Purpose: Owns outbox event polling, monitoring, and replay.

Main API groups:

- /api/v1/queue
- /health

Important behavior:

- Reads pending events from outbox_events.
- Marks events done or failed.
- Tracks retry count and error message.
- Supports replay of failed events.

7.8 Search Service

Location: search-service

Purpose: Owns unified search across service domains.

Main API groups:

- /api/v1/search
- /health

Important behavior:

- Accepts a query and optional limit.
- Searches tickets, assets, agents, and policies.
- Returns grouped results for the frontend search page.

Section 8

Data Model Summary

Major tables/entities visible from the documented codebase include:

Domain	Important Entities
Identity	User, Organization, role fields, privilege/clearance metadata
Notifications/Activities	Notification, ActivityLog, reminders
Assets/Devices	Asset, Device, EnrollmentToken, DeviceCommand
Tickets	Ticket, TicketComment, TicketAttachment, TicketLink, TicketSLA, OutboxEvent
Directory	Agent, Availability, OnCallSchedule, Metrics
Policy/Security	Policy, PolicyAssignment, DeviceGroup, DeviceSecurityProfile, SecurityAlert, RemediationAction, DeviceCommand
Automation	AutomationRule, AutomationExecution
Files	File metadata/index entries
Queue	OutboxEvent

The current Docker setup uses a shared PostgreSQL database during the microservice transition. This keeps local development simple but means schema ownership must be handled carefully.

© 2026 RiskTrix · Confidential — Internal Use Only

Section 9

Key Cross-Service Workflows

9.1 Ticket Auto-Assignment

Why does automatic assignment matter?

Without auto-assignment, someone would have to manually read every new ticket and decide who to give it to. This is slow and creates bottlenecks. The auto-assignment system uses live workload data from the Agent Directory to instantly find the least-busy person who has the right skills for that type of ticket, and assigns it automatically the moment the ticket is created.

1. User creates a ticket.
2. Ticket Service determines assignment requirements.
3. Directory Service returns matching active agents by team/tier.
4. Ticket Service checks workload.
5. Ticket is assigned to the least-loaded suitable agent.
6. Notification Service sends/logs assignment email.
7. Ticket Service records events for queue processing.

9.2 Policy Compliance and Security Risk

The continuous security feedback loop

This is not a one-time check — it is a continuous cycle. Policies define what secure looks like. Devices report their state when they check in. Compliance scans measure each device against the policies. The risk engine scores each device based on what it finds. Alerts are raised when something is wrong. Automation rules respond immediately. This loop runs constantly in the background so threats are caught quickly.

1. Policies are created and assigned to devices/groups.
2. Device/agent check-ins update device posture.
3. Compliance scans evaluate assigned policies.
4. Security module updates device risk profile.
5. Risk engine assigns risk score and risk level.
6. Alerts and remediation actions are created when required.
7. Automation rules may react to events.

© 2026 RiskTrix · Confidential — Internal Use Only

9.3 Outbox Queue Processing

1. A service writes an event to the outbox table.
2. Queue Service polls pending events.
3. Worker attempts delivery/handling.
4. Event becomes done or failed.
5. Queue Monitor lets privileged users inspect and replay failures.

Section 10

Deployment and Local Startup

The main deployment file is docker-compose.yml.

Common startup command:

```
docker compose up --build
```

Important URLs:

URL	Purpose
`http://localhost:8888`	Frontend
`http://localhost:8000/docs`	Main backend Swagger docs
`http://localhost:8003/docs`	Asset service Swagger docs
`http://localhost:8004/docs`	Ticket service Swagger docs
`http://localhost:8005/docs`	Directory service Swagger docs
`http://localhost:8006/docs`	Policy service Swagger docs
`http://localhost:8007/docs`	File service Swagger docs
`http://localhost:8008/docs`	Notification service Swagger docs
`http://localhost:8009/docs`	Queue service Swagger docs
`http://localhost:8010/docs`	Search service Swagger docs
`http://localhost:8080/health`	API gateway health

Important environment variables are shown in .env.example and docker-compose.yml, including database credentials, Redis URL, JWT secret, token expiry, service URLs, and mail settings.

Section 11

API Gateway Notes

The API gateway is implemented with Nginx in api-gateway/nginx.conf. Docker Compose starts the gateway along with the frontend, backend, microservices, PostgreSQL, and Redis.

In the current project, the frontend also uses module-specific API utilities and direct service URLs for broader service access.

Section 12

Operational Notes

- Use /health endpoints for service health checks where enabled.
- Some health checks are disabled in Docker Compose for services still being stabilized.
- Keep SECRET_KEY and database passwords out of production source control.
- PostgreSQL is shared across services in this local architecture; migrations should be reviewed before applying.
- Uploaded files are mounted into service-specific uploads directories.
- Redis is available for event/cache workflows, while queue monitoring also uses the database-backed outbox pattern.
- Auditor access is intentionally limited for operational modules.
- The sidebar contains duplicate ticket assignment labels pointing to /tickets/assignments.

© 2026 RiskTrix · Confidential — Internal Use Only

Section 13

Quick New-Developer Map

Start reading in this order:

1. docker-compose.yml to understand running services and ports.
2. cisodashboard-frontend/src/components/Navigation/Sidebar.jsx to understand user-visible modules.
3. cisodashboard-frontend/src/App.jsx to understand routes and guards.
4. cisodashboard-frontend/src/utils/permissions.js to understand roles.
5. cisodashboard-backend/app/api/v1/api.py to understand main backend route composition.
6. Each documented microservice main.py to understand service startup and included routers.
7. Each documented service app/api/routes folder to inspect concrete endpoints.
8. Each documented service app/models and app/schemas folder to understand data structures.

© 2026 RiskTrix · Confidential — Internal Use Only

Section 14

Glossary

Term	Meaning
CISO	Chief Information Security Officer
POC	Proof of Concept / closure evidence file
SLA	Service Level Agreement
MDM	Mobile Device Management
RBAC	Role-Based Access Control
Outbox	Database table used to persist events before asynchronous delivery
Agent	Internal support/security user eligible for assignment or on-call work

© 2026 RiskTrix · Confidential — Internal Use Only

Section 15

Detailed Page-by-Page Feature Walkthrough

This section documents the actual pages, submodules, tabs, filters, tables, forms, dialogs, and actions implemented in the frontend for the modules shown in the screenshots and sidebar.

15.1 Asset Management

Asset Inventory Page

Route: /assets

Purpose: Main asset listing page for managing all registered assets.

Visible sections and controls:

- Header: Asset Inventory with subtitle Manage and monitor all registered assets.
- Action buttons:
- Import Excel: opens bulk import dialog for .xlsx, .xls, and .csv.
- Create Asset: opens create asset modal.
- Summary cards:
- Total Assets
- High Risk
- Medium Risk
- Low Risk
- Search field:
- Placeholder: Search assets...
- Searches by asset name.
- Risk filter chips:
- All
- High
- Medium
- Low
- Empty state:
- Shows No assets found.
- If no assets exist, shows Create your first asset.
- Asset table columns:
- Name

© 2026 RiskTrix · Confidential — Internal Use Only

- Type
- Risk
- Vulnerabilities
- Created
- Actions
- Row actions:
- View Detail
- Edit Asset
- Delete Asset

Create/Edit Asset fields:

- Asset Name
- Type:
- Web Application
- Mobile Application
- API
- Server
- Database
- Network Device
- Endpoint
- Cloud Service
- IoT Device
- Container
- Firewall
- Router
- Switch
- Load Balancer
- Risk Level:
- Low
- Medium
- High
- Business Criticality:
- Critical
- High
- Medium
- Low
- Tags
- Compliance Scope
- Vendor / Manufacturer
- Owner

- Custodian
- IP Address
- Hostname
- Purchase Date
- Warranty Expiry
- End of Life

Bulk import dialog:

- Title: Import Assets from Excel / CSV.
- Accepts .xlsx, .xls, and .csv.
- Supports drag-and-drop and file picker.
- Processes all sheets in an Excel workbook.
- Recognized flexible columns include:
- Device make & Model
- Serial Number
- U Space
- Power Source
- Power watts
- Type
- Risk Level
- Location
- Site
- Notes
- Result screen shows:
- Assets Added
- Skipped / Failed
- Import message
- Row-level errors

Asset Dashboard Page

Route: /assets/dashboard

Purpose: Dashboard summary for asset posture.

Main capabilities:

- Loads dashboard metrics from Asset Service.
- Shows asset totals and risk distribution.
- Uses asset summary cards.
- Displays asset type distribution.
- Displays asset risk chart information.

- Provides quick visibility into the asset estate without opening the full inventory table.

Asset Detail Page

Route: /assets/:id

Purpose: Full detail view for one asset.

Top summary:

- Asset name and identifier.
- Risk Level card.
- Asset Type card.
- Vulnerability count card.
- Warning panel when the asset has open vulnerabilities.

Tabs:

1. Information
2. Vulnerabilities
3. Software
4. Files
5. Change Log

Information tab fields:

- Asset Name
- Asset ID
- Type
- Risk Level
- Business Criticality
- Tags
- Compliance Scope
- Open Vulnerabilities
- Vendor
- Owner
- Custodian
- IP Address
- Hostname
- Purchase Date
- Warranty Expiry
- End of Life
- Serial Number
- Device Model

- Rack Position
- Power Source
- Power Watts
- Location
- Site
- Notes
- Created At

Vulnerabilities tab:

- Shows asset-specific vulnerabilities.
- Empty state: No vulnerabilities recorded for this asset.
- Table columns:
- CVE ID
- Title
- Severity
- CVSS
- Status
- Discovered
- Actions
- Actions:
- Add Vulnerability
- Edit Vulnerability
- Mark Resolved
- Delete Vulnerability
- Add/Edit Vulnerability fields:
- Title
- CVE ID
- Severity: critical, high, medium, low, info
- CVSS Score
- Status: open, in_progress, accepted, resolved
- Description
- Remediation Notes

Software tab:

- Shows installed software linked to the asset.
- Empty state: No software recorded for this asset.
- Table columns:
- Name
- Vendor
- Version

- Patch Level
- Outdated
- Licensed
- License Expiry
- Actions
- Actions:
- Add Software
- Remove Software
- Add Software fields:
- Name
- Vendor
- Version
- Patch Level
- Outdated?
- Licensed?

Files tab:

- Uses File Upload Zone with context asset.
- Lists files attached to the asset reference ID.
- Supports upload, download, and delete based on permission.

Change Log tab:

- Shows field-level asset changes.
- Empty state: No changes recorded yet.
- Table columns:
- When
- Changed By
- Field
- Old Value
- New Value

15.2 Policies

The Policies sidebar section contains:

- Policy Dashboard
- Policy Management
- Policy Assignments
- Bulk Assign
- Conflict Report

- Policy Templates
- Compliance Dashboard
- Device Groups
- Devices
- Enrollment Dashboard
- Enroll Device

Policy Dashboard

Route: /policies/dashboard

Purpose: Multi-audience policy health view.

Header actions:

- Bulk Assign
- Policies
- Refresh

Top KPI cards:

- Active Policies
- Policy Coverage
- Pending Actions
- High-Risk Devices

Dashboard tabs:

1. IT Operations
2. Security
3. Compliance
4. Executive

IT Operations tab:

- Shows total assignments, successful deployments, failed deployments, pending deployments, retrying deployments, and success rate.
- Displays deployment progress bars.
- Shows recently failed deployments with device, policy, and failure reason.
- Links device names to device detail pages.
- Links policy names to policy detail pages.

Security tab:

- Shows security-oriented policy posture.
- Surfaces risky devices and policy/security failures.
- Helps identify devices that need attention.

© 2026 RiskTrix · Confidential — Internal Use Only

Compliance tab:

- Shows compliance metrics and scan results.
- Helps determine whether assigned policies are passing or failing.

Executive tab:

- Summarizes policy coverage, risk, and pending action counts for leadership-level review.

Policy Management

Route: /policies

Purpose: Create, search, filter, view, edit, and delete policies.

Visible sections:

- Header: Policy Management.
- Subtitle: Define and manage device security policies.
- Create Policy button.
- Summary cards:
- Total Policies
- Active
- Draft
- Search field:
- Placeholder: Search policies...
- Status filter chips:
- All
- Active
- Draft
- Archived
- Policy table columns:
- Name
- Category
- Platform
- Status
- Created
- Actions
- Row actions:
- View
- Edit
- Delete

Create Policy modal:

- Policy Name
- Description
- Category:
- Device Security
- App Control
- Network
- OS Update
- Platform:
- All Platforms
- Android
- Windows
- iOS
- macOS
- Status:
- Active
- Draft
- Device security controls:
- Require Screen Lock
- Password Complexity
- Encryption Required
- Min Password Length
- Password Expiry days
- Connectivity controls:
- Disable Bluetooth
- Disable USB
- Restrict Wi-Fi
- OS update controls:
- Require OS Update
- Minimum OS Version

Policy Detail

Route: /policies/:id

Purpose: Inspect one policy, its settings, assigned devices, and controls.

Key areas:

- Policy metadata and status.
- Assigned devices table.
- Actions for assignment retry, device navigation, and assignment removal.
- Description panel.

- Device Security section:
- Require Screen Lock
- Password Complexity
- Encryption Required
- Min Password Length
- Password Expiry
- Connectivity section:
- Bluetooth Disabled
- USB Disabled
- Wi-Fi Restricted
- OS Update section:
- OS Update Required
- Min OS Version

Policy Edit

Route: /policies/:id/edit

Purpose: Update an existing policy using the same categories and controls as the create policy form.

Policy Apply

Route: /policies/:id/apply

Purpose: Assign a selected policy to target devices and manage deployment behavior.

Policy Assignments

Route: /policies/assignments

Purpose: Monitor policy deployment and compliance assignments.

Visible sections:

- Header: Policy Assignments.
- Refresh action.
- Navigate to Policies action.
- Summary/stat cards for assignment status.
- Deployment success rate.
- Compliance health rate.
- Audit event list.

Audit event table columns:

- Action

- Policy
- Device
- Performed By
- Details
- Time

Bulk Assign

Route: /policies/bulk-assign

Purpose: Wizard for assigning multiple policies to multiple devices in one operation.

Wizard steps:

1. Select Policies
2. Select Devices
3. Review & Deploy
4. Success screen

Select Policies step:

- Search policies.
- Select all visible policies.
- Select individual policy rows.
- Shows policy name, category, platform, and status.

Select Devices step:

- Search devices.
- Select all visible devices.
- Select individual device rows.
- Shows device name, owner, OS version, and platform.

Review & Deploy step:

- Shows total number of assignments that will be created.
- Shows selected policies.
- Shows selected devices.
- Deploy button runs the bulk assignment.

Success screen:

- Shows Bulk Deployment Complete.
- Shows total assignments, successful, failed, and skipped.
- Shows deployment success rate.
- Shows assignment result table with:
- Policy ID

© 2026 RiskTrix · Confidential — Internal Use Only

- Device ID
- Deployment
- Compliance
- Score
- Provides New Bulk Assignment and View Assignments actions.

Conflict Report

Route: /policies/conflicts

Purpose: Show policy conflicts detected across device assignments.

Main functionality:

- Lists policies that may contradict each other.
- Groups conflicts by affected devices.
- Helps administrators reduce contradictory device controls.

Policy Templates

Route: /policies/templates

Purpose: Browse built-in policy templates and apply them as real policies.

Main functionality:

- Lists template policies.
- Shows category/platform/template metadata.
- Allows applying a template to create a policy.

Compliance Dashboard

Route: /compliance

Purpose: Show policy compliance scan results and compliance posture.

Main functionality:

- Displays compliance score and status distribution.
- Shows compliant, non-compliant, and warning states.
- Supports scan/refresh style compliance workflows.
- Links compliance status back to assignments/devices where available.

Device Groups

Route: /groups

Purpose: Manage static and smart groups of devices.

Visible sections:

- Header: Device Groups.
- Refresh button.
- Create Group button.
- Empty state when no groups exist.
- Group cards/table entries showing group name, type, rule/members, policy count, and device count.

Create Device Group dialog:

- Group type toggle:
- Static Group
- Smart Group
- Group Name
- Description
- Smart group rule:
- Field
- Operator
- Value

Smart group fields/operators are used to auto-match devices based on device metadata.

Device Group Detail

Route: /groups/:id

Purpose: Inspect one device group, its members, and assigned policies.

Main functionality:

- Shows group metadata.
- Lists member devices.
- Adds/removes members for static groups.
- Shows policies assigned to the group.
- Supports assigning/removing group policies.

Devices

Route: /devices

Purpose: Device inventory for policy/security management.

Main functionality:

- Lists enrolled/managed devices.
- Shows platform, owner, OS version, online/offline status, and risk/compliance indicators.

- Links to device detail.

Device Detail

Route: /devices/:id

Purpose: Inspect one device's policy assignments, violations, activity, and commands.

Main functionality:

- Shows device information.
- Shows assigned policies.
- Shows violations.
- Shows activity logs.
- Supports command-related actions where available.

Enrollment Dashboard

Route: /enrollment

Purpose: Overview of device enrollment pipeline.

Main functionality:

- Shows enrollment statistics.
- Shows enrollment history.
- Shows token-related enrollment information.

Enroll Device

Route: /enrollment/enroll

Purpose: Form workflow for registering/enrolling a new device.

Main functionality:

- Captures device details.
- Creates device enrollment records.
- Connects device enrollment to asset/device management.

15.3 Tickets

The Tickets sidebar section contains:

- All Tickets
- Ticket Dashboard
- Ticket Assignments

- Assignments

All Tickets

Route: /tickets

Purpose: Main incident ticket management page.

Header actions:

- Export CSV
- Create Ticket

Stats cards:

- Total
- Open
- Resolved
- Closed
- Critical
- SLA Breached

Search and filters:

- Search by title, ticket number, or caller.
- Status filters:
- All
- New
- Assigned
- In Progress
- Resolved
- Closed
- Priority filters:
- All Priority
- Critical
- High
- Medium
- Low

Bulk actions:

- Select individual tickets.
- Select all filtered tickets.
- Bulk Close.
- Bulk Reassign.

- Update Status.
- Clear selection.

Ticket table columns:

- Ticket #
- Title & Description
- Caller
- Priority
- Status
- Category
- Assigned To
- Created
- Updated
- Actions

Table behavior:

- SLA-breached rows are visually highlighted.
- Ticket number shows SLA breach warning icon when response or resolution SLA is breached.
- Updated column shows relative time.
- View action opens ticket detail.

Create Ticket modal:

- Caller Name
- Caller Email
- Channel:
- Self-Service
- Phone
- Email
- Chat
- Walk-In
- Asset ID
- Short Description / Title
- Description
- Category:
- General
- Security
- Network
- Hardware
- Software
- Subcategory

© 2026 RiskTrix · Confidential — Internal Use Only

- Impact:
- High
- Medium
- Low
- Urgency:
- High
- Medium
- Low
- Assignment Group
- Assigned To
- Created By
- Calculated Priority preview based on Impact + Urgency

Bulk Reassign dialog:

- Assign To
- Assignment Group

Bulk Update Status dialog:

- New Status
- Shows transition guidance for invalid or skipped transitions.

Ticket Dashboard

Route: /tickets/dashboard

Purpose: Analytical dashboard for ticket operations.

Top actions:

- Refresh.
- Navigate to Tickets.

Dashboard KPIs:

- Total Tickets
- Open Tickets
- Resolved
- Closed
- SLA Breached
- SLA Compliance

Visual sections:

- Status breakdown chart.
- Priority breakdown chart.

- Category/channel-oriented distributions where data is available.
- SLA compliance visualization.
- Recent tickets table.

Recent tickets table includes:

- Ticket number
- Title
- Priority
- Status
- Assigned To

Ticket Assignments / Assignments

Route: /tickets/assignments

Purpose: Workload and assignment dashboard for ticket operations.

Header actions:

- Refresh
- Back to Tickets

Summary cards:

- Total assigned workload
- Open workload
- Unassigned open tickets
- Team/person workload indicators

Tabs:

1. By Team
2. By Person
3. Unassigned

By Team table columns:

- Team / Group
- Total
- Open
- Resolved
- Closed
- Priority Breakdown
- Workload

By Person table columns:

- Person
- Team / Group
- Total
- Open
- Resolved
- Closed
- Priority Breakdown
- Workload

Unassigned tab:

- Lists unassigned open tickets that need routing.
- Shows priority, status, category, and created time.
- Links back into ticket detail.

Ticket Detail

Route: /tickets/:id

Purpose: Full ticket record page.

Top area:

- Back button.
- Ticket title and ticket number.
- Priority chip.
- Status chip.
- SLA response and resolution cards.
- Status transition buttons where allowed.

Tabs:

1. Details
2. Notes
3. Resolution Information
4. Related Records
5. Files
6. Change History

Details tab:

- Shows description and key metadata.
- Shows caller/assignee/assignment group/category details.
- Shows Assign Ticket section when ticket status is new.
- Assign Ticket fields:

- Assign To
- Assignment Group

Notes tab:

- Lists existing notes/comments with author and timestamp.
- Add Note form:
- Note
- Your Name
- Post Note

Resolution Information tab:

- If ticket is in progress, shows resolution form:
- Resolution Code
- Resolved By
- Resolution Notes
- Mark as Resolved
- If already resolved/closed, shows:
- Resolution Code
- Resolved By
- Resolved At
- Resolution Notes
- If not ready for resolution, shows message explaining that the ticket must move to In Progress.

Related Records tab:

- Ticket Timeline:
- Ticket Created
- Assigned
- In Progress
- Resolved
- Closed
- Key Dates:
- Created At
- Last Updated
- Resolved At
- Closed At

Files tab:

- File upload zone with context ticket.
- File list for files attached to the ticket reference.
- Supports download and delete where allowed.

© 2026 RiskTrix · Confidential — Internal Use Only

Change History tab:

- Field-level change history.
- Columns:
- Field
- Old Value
- New Value
- Changed By
- When

15.4 Security

The Security sidebar section contains:

- Security Dashboard
- Submit Agent Report

Security Dashboard

Route: /security

Purpose: Device risk overview, threat alerts, and security posture.

Header action:

- Re-evaluate All

Stat cards:

- Total Evaluated
- Critical Risk
- High Risk
- Open Alerts
- Security Score
- Risky Devices

Charts/widgets:

- Threat Alert Trend - Last 7 Days
- Top Security Issues:
- Rooted
- Unencrypted
- No Screen Lock
- OS Outdated
- Blacklisted App

Threat Alerts section:

- Alert count chip.
- Filters:
- Severity: All, Critical, High, Medium, Low
- Status: All, Open, Resolved, Suppressed
- Alert Type: All, Rooted Device, Encryption Disabled, No Screen Lock, OS Outdated, Blacklisted App
- Device ID
- Actions:
- Apply filters
- Clear filters
- Table columns:
- Device ID
- Alert Type
- Severity
- Message
- Time
- Status
- Actions
- Alert row actions:
- Acknowledge/suppress style action depending on current state.
- Resolve alert.
- Navigate to device security detail.

Risky Devices table:

- Columns:
- Device
- Owner
- Platform
- Risk Score
- Risk Level
- Main Issue
- Quick Actions
- Row click opens device security detail.
- Quick actions include common remote command shortcuts such as lock and sync.

Submit Agent Report

Route: /security/report

Purpose: Manual form for submitting device security scan results.

Main sections:

- Device Information:
- Device ID
- Risk preview:
- Risk Score out of 120
- Risk level chip
- Number of detected issues
- Action buttons:
- Submit report
- Reset
- Security Flags:
- Rooted/Jailbroken
- Encryption disabled
- Screen lock disabled
- OS outdated
- Blacklisted app detected
- Each flag has:
- Description
- Risk point weight
- ON/OFF switch

Submit behavior:

- Sends profile data to security API.
- Backend recalculates risk score and risk level.
- Result screen shows new risk score and risk level.

Device Security Detail

Route: /security/device/:id

Purpose: Deep inspection of one device's security profile, alerts, remediation actions, heartbeat, and remote commands.

Top section:

- Back to Security.
- Device security title.
- Re-evaluate button.

Risk Score card:

- Shows score out of 120.
- Shows risk level.

- Shows last evaluated timestamp.

Security Flags card:

- Root / Jailbreak
- Encryption
- Screen Lock
- OS Up-to-date
- No Blacklisted Apps

Device Heartbeat card:

- Online/offline status.
- Last Seen.
- Network.
- Agent version.
- OS Version.
- Battery.
- Storage.

Alerts table:

- Shows alerts for the selected device.
- Displays alert type, severity, message, status, and timestamps.

Remediation section:

- Create/log remediation action:
- Action Type
- Notes
- Performed By
- Remediation table columns:
- Action Type
- Performed By
- Notes
- Status
- Created
- Completed
- Update
- Status update actions:
- Mark In Progress
- Mark Completed
- Mark Failed

Remote Actions panel:

- Commands are queued and executed when the device checks in.
- Actions:
- Lock Device
- Factory Wipe
- Restart
- Force Sync
- Send Message
- Clear Passcode
- Lost Mode
- Confirmation/input dialogs:
- Dangerous commands require confirmation.
- Send Message requires message text.
- Command History table:
- Command
- Issued By
- Status
- Sent At
- Cancel
- Pending/sent commands can be cancelled.

15.5 Automation

The Automation sidebar section contains:

- Automation Rules
- New Rule

Automation Rules

Route: /automation

Purpose: Automation engine dashboard for rules and executions.

Top section:

- Header: Automation Engine.
- New Rule button.
- Error alerts when API calls fail.

Stats cards:

- Total rules.
- Enabled rules.

- Execution counts.
- Success/failure/skipped counts where available.

Rules table columns:

- Rule
- Trigger
- Conditions
- Actions
- Enabled
- Last Updated
- Actions

Rule row actions:

- Test rule.
- Edit rule.
- Delete rule.
- Enable/disable switch.

Execution history table columns:

- Rule
- Trigger
- Device
- Actions Taken
- Status
- Executed At

Delete Rule dialog:

- Confirms deletion.
- Provides Cancel and Delete actions.

Test Rule dialog:

- Shows selected rule.
- Runs test with sample context.
- Displays whether conditions matched.
- Displays actions that would run.
- Shows error/success/warning result state.

New Rule / Edit Rule

Routes:

- /automation/new

- /automation/:id/edit

Purpose: Build or edit an automation rule.

Rule Details section:

- Rule Name
- Enabled/Disabled switch
- Description

Trigger section:

- Dropdown: When to evaluate this rule.
- Supported trigger categories include:
- Risk evaluated
- Device offline
- Agent check-in
- Manual/test-oriented trigger

Conditions section:

- Title: Conditions (ALL must match).
- Add Condition button.
- Each condition has:
- Field
- Operator
- Value
- Remove action
- If no conditions exist, the rule can match all trigger events.

Actions section:

- Title: Actions (all fire in order).
- Add Action button.
- Each action has:
- Action Type
- Additional fields depending on action type
- Remove action
- Supported action configuration fields include:
- Severity
- Alert Type
- Remediation Action Type
- Message
- Notes

Footer actions:

- Cancel
- Create/Save Rule

15.6 File Manager

Route: /files

Purpose: Central file upload, browsing, download, and deletion.

Header:

- Title: File Manager.
- Refresh button.

Summary cards:

- Total Files
- Total Size
- Counts by context.

Upload File section:

- Context selector chips:
- general
- ticket
- vulnerability
- poc
- audit
- asset
- File Upload Zone:
- Shows context.
- Shows reference ID when supplied.
- Supports click or drag-and-drop upload.
- Shows upload progress.

All Files section:

- Search field:
- Placeholder: Search files...
- Context filter chips:
- All
- general
- ticket

- vulnerability
- poc
- audit
- asset
- File table columns:
- File Name
- Context
- Ref ID
- Size
- Type
- Uploaded
- Actions
- Row actions:
- Download
- Delete

15.7 Agent Directory

Route: /directory

Purpose: Manage support/security agents, on-call coverage, availability, workload, and performance metrics.

Top summary:

- Total Agents
- Active
- Senior Tier
- Open Tickets
- On Call Now

Header actions:

- Refresh
- Schedule On-Call
- Add Agent

Tabs:

1. Agents
2. On-Call Now
3. Performance Leaderboard

Agents tab:

- Search placeholder: Search name, email, team, skill...
- Filters:
- Team
- Tier: Senior, Standard
- Show inactive switch
- Clear filters
- Table columns:
- Agent
- Team
- Tier
- Skills / Categories
- Open Tickets
- Availability
- Status
- Actions
- Availability chips:
- On Call
- Available
- Off Hours
- Status chips:
- Active
- Inactive
- Actions:
- Edit Agent
- Delete Agent

Add/Edit Agent modal:

- Identity section:
- Full Name
- Email Address
- Assignment section:
- Assignment Team
- Experience Tier
- Skills / Categories:
- Comma-separated skills such as network, malware, phishing.
- Skills render as chips.
- Availability/status section:
- Active switch.
- Footer:

- Cancel
- Create/Save

On-Call tab:

- Header: On-Call Schedule.
- Schedule On-Call button.
- Empty state when no on-call entries exist.
- Table columns:
- Agent
- Team
- Type
- Status
- Start
- End
- Notes
- Actions
- Schedule statuses:
- Active Now
- Upcoming
- Past
- Schedule types:
- Primary
- Backup
- Actions:
- Delete on-call entry.

Schedule On-Call dialog:

- Agent
- Start Time
- End Time
- Type:
- Primary
- Backup
- Notes

Performance Leaderboard tab:

- Shows ranking of agents based on performance metrics.
- Empty state when no metrics exist.
- Table columns:
- Rank

- Agent
- Team
- Tickets
- Avg Resolution
- Quality Score
- SLA Compliance

15.8 Queue Monitor

Route: /queue

Purpose: Monitor outbox events and replay failed work.

Header:

- Title: Queue Monitor.
- Refresh button.

Summary cards:

- Total Events
- Pending
- Done
- Failed
- Processing

Filters:

- Status:
- All statuses
- Pending
- Processing
- Done
- Failed
- Row limit
- Apply
- Clear

Event table columns:

- ID
- Event Type
- Status
- Retries

- Created At
- Processed At
- Error
- Actions

Row actions:

- View event detail.
- Replay failed event.

Event Detail dialog:

- Event Type
- Status
- Retries
- Created At
- Processed At
- Error
- Payload
- Close button
- Replay button when the event is failed

15.9 How The Documented Modules Connect

Important end-to-end relationships:

- Tickets can link to assets by Asset ID.
- Ticket detail and asset detail both use File Manager through contextual uploads.
- Ticket auto-assignment uses Agent Directory team/tier and workload data.
- Policy Service uses devices, groups, assignments, compliance scans, security profiles, alerts, remediation, commands, and automation.
- Security Dashboard and Device Security Detail use remote command APIs also used by policy/device command features.
- Queue Monitor observes asynchronous outbox work created by backend services.
- Automation can react to security and device events and create alerts, remediation notes, notifications, or ticket-related actions depending on configured rule actions.

Section 16

Conclusion

RiskTrix is organized around business modules visible in the sidebar and backed by a combination of the main backend and specialized microservices. In this version, the documentation focuses only on the modules and services in scope, excluding the areas built by someone else.

© 2026 RiskTrix · Confidential — Internal Use Only